

FAST – NUCES



Object Oriented Programming Lab Manual

Course Teacher	Ms. Hina Iqbal
Lab Instructor	Mr. Ahmad Jawad Mustasim
Lab TA	Ms. Mania Shakeel
Section	BSE-2B
Semester	Spring 2026

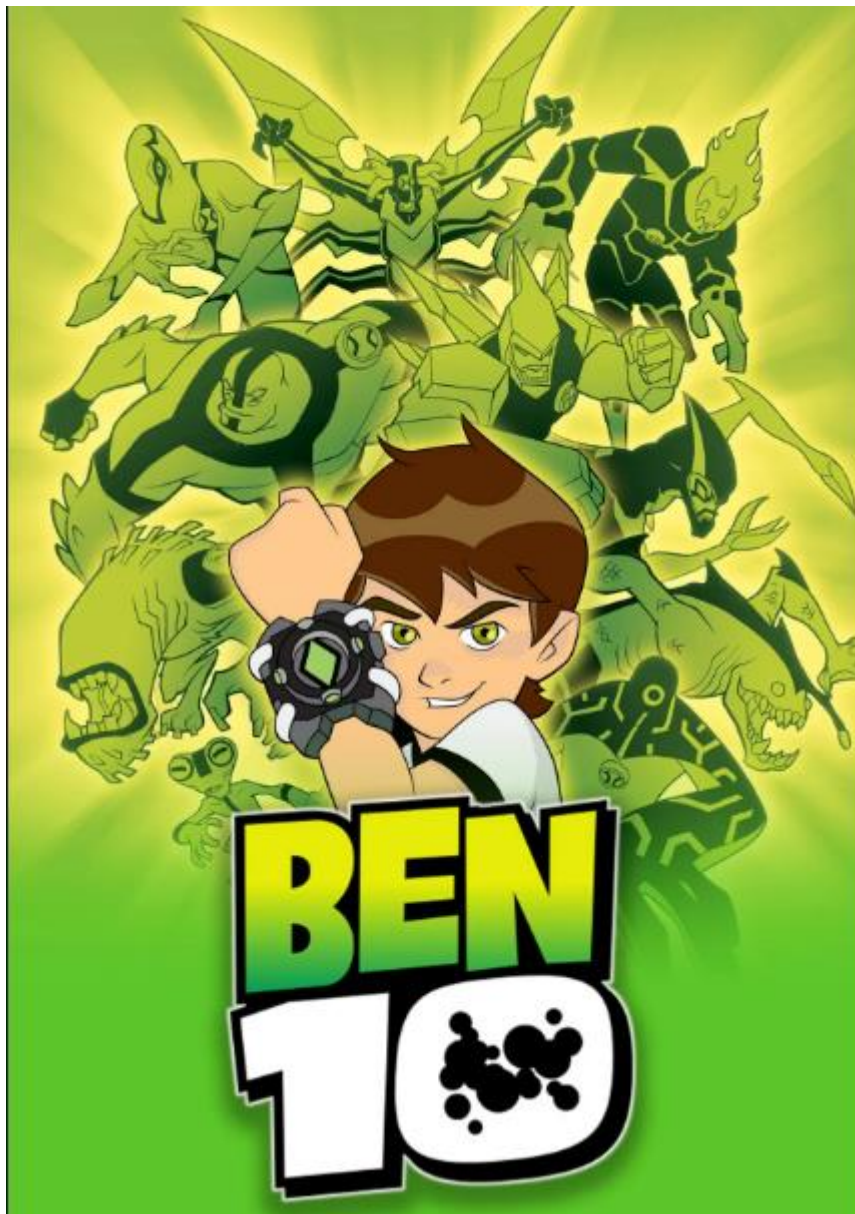
Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab 13 – Diamond Problem in Inheritance and Exception Handling

Objectives

- Implementation of Multiple Inheritance
 - Hybrid Inheritance Example
 - Understanding example of diamond problem
 - Solving diamond problem using virtual inheritance
 - Introduction to Exception Handling
-



Read carefully and attempt following task

Multiple Inheritance & Diamond Problem (Human–Alien Hybrid Theme)

In a futuristic Omniverse, beings can access **human intelligence** and **alien powers** simultaneously through a device called the **Omnitrix**.

We'll simulate this concept using four classes:

- **Character** — Base class (common identity attributes).
- **Human** — Derived from Character, represents logical, emotional traits.
- **Alien** — Derived from Character, represents powerful, otherworldly traits.
- **HumanAlienCombo** — Derived from both Human and Alien, representing a complete hybrid who wields an **Omnitrix**.



Abstraction of Code (use separate files):

Use separate header and cpp file for each class. Implement abstraction.

Input Validation Using Exception Handling

To make the system robust and realistic, all setter methods must include **input validation using exception handling**.

Rules:

- Name must not be empty
- Age must be ≥ 0
- Strings like nationality, profession, planetOrigin, specialPower must not be empty
- Omnitrix ID must be positive

Example:

```
void setAge(int age) {  
    if (age < 0) {  
        throw invalid_argument("Age cannot be negative!");  
    }  
    this->age = age;  
}
```

This ensures invalid data is never assigned to objects.

Instructions:

Before implementing the classes, ensure the following:

1. Encapsulation

- All attributes must be private
- Access via getters/setters only

2. Exception Handling in Setters

- Every setter must validate input
- Use try-catch in main() to handle errors

3. Validation Rules

- Empty strings - throw exception
- Negative values - throw exception
- Invalid IDs - throw exception

Example Pattern:

```
try {  
    obj.setAge(-5);  
} catch (const exception& e) {  
    cout << "Error: " << e.what() << endl;  
}
```

Classes Design:

1. Class Character

Purpose: The most general base class for all beings.

Attributes:

- string name
- int age

Functions:

- Default and parameterized constructors
- void displayCharacterInfo()
- Getters and setters for both name and age

2. Class Human (Derived from Character)

Purpose: Represents the human side — intelligent, emotional, and creative.

Attributes:

- string nationality
- string profession

Functions:

- Constructors (pass to Character)
- void displayHumanTraits()
- Getters and setters for all attributes
- Example behavior method:
void think() - prints something like “uses logic and human intelligence.”

3. Class Alien (Derived from Character)

Purpose: Represents the alien side - powers and origin.

Attributes:

- string planetOrigin
- string specialPower

Functions:

- Constructors (pass to Character)
- void displayAlienTraits()
- Getters and setters for both attributes
- Example behavior method:
void usePower() - prints something like “activates Plasma Beam attack!”

4. Class **HumanAlienCombo** (Derived from **Human** and **Alien**)

Purpose: Represents a being that combines human intelligence and alien powers via the **Omnitrix**.

Attributes:

- int omnitrixID - unique ID for the Omnitrix device
- string omnitrixVersion - identifies which model of Omnitrix is used (e.g., “V1”, “Ultimate”, “Omniverse”)

Functions:

- Constructors (initialize all base and new members)
- void displayHybridProfile() - shows full info (human + alien + Omnitrix)
- Getters and setters for new attributes
- Example behavior methods:
 - void activateOmnitrix() - print that the Omnitrix is activated.
 - void displayAbilities() - call both think() and usePower() to show combined abilities.

Main Implementation Requirements:

1. Implement three scenarios in the main() function:
 - A Human-only object — showing human traits and behaviors.
 - An Alien-only object — showing alien traits and powers.
 - A Ben10Hybrid object — combining both (causing diamond problem).
 2. Include full encapsulation:
 - Use private data members.
 - Implement getters and setters (with exceptional handling) for all attributes.
 3. Demonstrate the diamond problem:
 - First, inherit normally (without virtual) - ambiguity appears when calling displayCharacterInfo().
 - Then, fix it by using virtual public Character in both Human and Alien.
-

4. Call Methods:

- displayCharacterInfo()
- displayHumanTraits() and think() for Human
- displayAlienTraits() and usePower() for Alien
- displayHybridProfile() and showCombinedAbilities() for Ben10Hybrid

Example Program Flow (Expected Output)

Human-Only Example

=== Human Information ===

Name: Ben Tennyson

Age: 16

Nationality: American

Personality: Brave

Ben Tennyson uses logic and human intelligence to solve problems.

=== Human Information ===

Name: Gwen Tennyson

Age: 16

Nationality: American

Personality: Intelligent

Gwen Tennyson uses logic and strategic thinking to solve problems.

Alien-Only Example

=== Alien Information ===

Name: Heatblast

Age: 200

Planet of Origin: Pyros

Special Power: Fire Manipulation

Heatblast uses Fire Manipulation to unleash a blazing attack!

=== Alien Information ===

Name: XLR8

Age: 150

Planet of Origin: Kinet

Special Power: Super Speed

XLR8 uses Super Speed to move faster than the eye can see!

Hybrid Example (Multiple Inheritance)

=== Ben10 Hybrid Profile (Fixed Diamond) ===

Name: Ben Tennyson Hybrid

Age: 16

Nationality: American

Personality: Brave

Planet Origin: Pyros

Special Power: Fire Manipulation

Omnitrix ID: 1001

Omnitrix Version: V2.5

Ben Tennyson uses logic and human intelligence to solve problems.

Ben Tennyson unleashes Fire Manipulation power!

Omnitrix activated - transformation sequence initiated!

-----Happy Coding! 😊-----



Submission Instructions

- Submit your tasks on Google Classroom within given deadline
 - Also submit screenshots of each task for example “LAB01-TASK01.png”, “LAB01-TASK02.png” and so on
 - Submit all .h and .cpp files
-

